

Executive Summary

This guide outlines a **step-by-step, time-boxed plan** to prepare for the HackerRank Orchestrate (Sep 2026, financial-agent) challenge as a first-time participant. It covers understanding the problem, inspecting the data, designing deterministic calculations, integrating AI for unstructured inputs, defining system architecture, implementing and testing the solution, and preparing for evaluation and interviews. We include **estimated times, deliverables, priorities, and risks for each step**, tables comparing architectural choices, example formulas/pseudocode, prompts, and a Gantt timeline. The focus is on building a robust system: **deterministic code for core finance logic** and **LLMs only for interpretation/explanation**, as advised by HackerRank experts [26†L131-L140] [34†L165-L169] .

1. Understand the Problem Statement

- **Time:** ~1–2 hours (high priority)
- **Deliverables:** Concise summary of requirements; list of required outputs (safe amount, status, method, plan, date, advice, explanation).
- **Risks:** Misreading requirements; missing key output fields (e.g. “earliest date”, “flexible spending”).

Action: Carefully read `problem_statement.md` and any sample outputs. Note exactly what outputs are expected for each request:

- **Max Safe Amount:** Highest payment today without breaking budget.
- **Affordability Status:** One of `affordable_now`, `affordable_with_plan`, `affordable_later`, `not_affordable`.
- **Payment Method:** One of `full_payment`, `partial_payment`, `installments`, `wait`, `not_recommended`.
- **Payment Plan:** If installments, details (months, amount).
- **Earliest Date:** When full payment is possible.
- **Flexible Spending Advice:** Categories to cut.
- **Explanation:** Short rationale for the decision.

Make a bullet list or table of these fields with definitions. HackerRank emphasizes *system design and reasoning* over naive LLM answers [26†L72-L80] . **Clarify assumptions:** e.g., what constitutes “essential” vs “flexible” expenses, how currencies are handled, etc.

2. Inspect and Map the Dataset

- **Time:** ~2–3 hours (high priority)
- **Deliverables:** Data inventory (list of files and key columns); notes mapping each column to decision inputs.
- **Risks:** Overlooking a file; misinterpreting column meaning (e.g. sign of amounts, currency codes).

Action: Examine the provided dataset files (using a tool like pandas or Excel). Typical files include:

- `financial_profiles.csv` : User profiles (ID, current balance, minimum balance requirement, currency).
- `financial_events.csv` : Scheduled transactions (salary, bills, rent, loans, etc.) with dates and amounts.
- `exchange_rates.csv` : Currency rates (to convert multi-currency values into a common currency for computation).
- `request_payment_options.csv` : For each request, lists allowed payment methods (full, partial, installment plans, with terms and amounts).
- `requests.csv` : Purchase requests (user ID, request ID, cost, currency, date).
- `messages.csv` : Unstructured text messages that may contain financial hints (e.g. "your rent is due").
- `images.csv` + `media/images/` : References to images (e.g. photos of bills) with info (e.g. upcoming expense amounts or deadlines).
- `sample_requests.csv` : Example solved rows (with expected outputs) to understand format.
- `output.csv` : Blank template for output (verify column order).

Document each file's schema. For each CSV, note columns, data types, and how they feed into decisions. For example, columns in `financial_events.csv` might include `user_id`, `date`, `amount`, `type` (salary, rent, utility, etc.), and you'll map those to incomes vs expenses in forecasting.

Tip: As suggested in Orchestrate workflows, **inspect the sample output** (or example rows) to confirm column names and format 【10†L259-L267】. Ensure you know the exact column order and names for `output.csv` (they must match exactly 【10†L328-L333】).

3. Design the Deterministic Financial Engine

- **Time:** ~4–6 hours (highest priority)
- **Deliverables:** Documented formulas/pseudocode for safe amount, forecasting, currency conversion, and categorization logic.
- **Priority:** High – core of correctness.
- **Risks:** Arithmetic bugs; ignoring future obligations; off-by-one errors in date logic; currency miscalculation.

3a. Safe Spending Calculation: Compute how much the user can spend **today** without violating budget rules. For each user:

- Let `balance` = current balance (converted to request currency).
- Subtract `min_balance` (the required safety buffer from `financial_profiles.csv`).
- Subtract **all upcoming essential obligations** (rent, utilities, loan payments, etc.) due *before* the purchase date. Also subtract any **pending debits**.
- **Formula (pseudocode):**

```
safe_now = balance - min_balance - sum(essentials_due_today_or_soon)
if safe_now < 0: safe_now = 0
```

This yields the **max safe amount** usable today. *For example*, if balance=₹80,000, essentials this month=₹30,000, min_balance=₹15,000, then safe_now=₹35,000. The user should not spend more than that now.

3b. Cashflow Forecasting: Simulate future balances to see when the full purchase can be paid. Walk through future dates (by day or month):

- Add scheduled incomes (salary, reimbursements) on their dates.
- Subtract scheduled bills/expenses on their due dates.
- Maintain the **min_balance** constraint at all times.
- Find the earliest date **T** when `(future_balance on T) - min_balance >= request_amount`.
If such T exists, set **earliest safe date = T**, else mark not affordable later.

Example pseudocode:

```
balance = current_balance
events = sorted future events (by date)
for date, amount, type in events:
    if date > today:
        balance += (amount if type=="income" else -amount)
        if balance - min_balance >= request_amount:
            earliest_date = date
            break
```

Keep currency consistent using `exchange_rates.csv` (e.g. convert all values to the request's currency or a base currency) to avoid errors.

3c. Affordability Status: Using **safe_now** and forecasts:

- If `safe_now >= request_amount`, then `affordable_now` and method= `full_payment`.
- Else, check **payment options**: if a *partial payment* option exists such that paying that partial amount now ($\leq$ safe_now) and paying the rest later fits the future cash flow, then status= `affordable_with_plan` and method= `partial_payment`.
- Else, if an *installment plan* (from `request_payment_options.csv`) can be serviced by the user's cashflow (i.e. each installment $\leq$ monthly disposable income), then status= `affordable_with_plan` and method= `installments`, choosing the longest/lowest monthly plan that works.
- Else if purchase can't be done now or via plan, but **can be fully done at a future date T** (computed above), set status= `affordable_later` and method= `wait`.
- Otherwise, status= `not_affordable` and method= `not_recommended`.

Always record the **payment plan** details (months and payment amount) or leave blank if not used.

3d. Flexible Spending Advice: If resources are tight, identify non-essential budget items to cut. Sum all "flexible" spending categories (e.g. entertainment, dining) from `financial_profiles.csv`. If `request_amount` exceeds safe funds, recommend reducing the largest flexible categories first. (E.g.

"Reduce dining ₹X and shopping ₹Y to afford this purchase.") This helps meet requirements for giving actionable advice.

Important: Perform all these calculations in code (Python/pandas), not by asking the LLM, to guarantee accuracy [26†L131-L140] . Deterministic rules are needed for safety and explainability.

4. Integrate AI for Unstructured Data

- **Time:** ~4 hours (medium-high priority)
- **Deliverables:** Modules or functions that extract financial info from text/images; example prompts and OCR results.
- **Risks:** Model hallucinations; OCR inaccuracies; missing crucial info hidden in text.

4a. Message Parsing (LLM/Regex/NER): For each row in `messages.csv`, run a routine to detect any transaction-related information. For example, use regex or a small LLM prompt to extract (amount, date, context).

- *Option 1:* Use regular expressions to find currency amounts and dates. E.g. pattern for "\$5000 on Sept 30" or "₹20000 rent".
- *Option 2 (LLM):* Use a prompt like:

*"Analyze this message and extract any financial events (type, amount, date):
{message_text}."*

This can capture nuanced info the regex misses.

4b. Image OCR: For each image in `media/images/` (paths given in `images.csv`), run OCR (e.g. Tesseract) to get text. Then apply the same extraction logic. Python example:

```
from PIL import Image
import pytesseract

text = pytesseract.image_to_string(Image.open(image_path))
# Then parse text for currency, dates
```

Alternatively, an API like Azure OCR could be used. The goal is to treat image info like extra messages.

4c. Integration: Any data extracted (e.g. a hidden salary note or bill amount) should be fed into the existing financial calculations: e.g. as an additional `financial_event`. The LLM itself should not be used to make final financial decisions, only to interpret messy input.

Example Prompt: If using GPT:

Message: "Your pending loan payment of \$1500 is due 2026-09-20."
Prompt: "Extract any financial amounts and due dates from the above message."

Expected Output: {"loan_payment": 1500, "due_date": "2026-09-20", "currency": "USD"}

5. Define Code vs. LLM Responsibilities

- **Time:** ~1–2 hours (high priority conceptual clarity)
- **Deliverables:** Written guidelines for what logic is coded vs what LLM handles.
- **Risks:** Overloading the LLM (error prone); leaving too much to code (inflexible).

Rule: “Let the model describe, let deterministic code decide” [34†L165-L169] . In practice:

- **Deterministic Code:** All arithmetic, comparisons, date math, currency conversion, rule enforcement, loops, and any exact computations (safe amount, forecasting, comparisons to thresholds) must be hard-coded. This ensures consistency and safety. For example, *never rely on an LLM to compute “80,000 – (20,000 + 15,000)” correctly*; do that in code.
- **LLM:** Only handle **unstructured text understanding** (parsing messages, generating the human-readable explanation) and optional plan presentation. For explanation generation, feed the **calculated results** into a prompt: e.g., “Given that the user can safely pay up to ₹35,000 today and the laptop costs ₹70,000, generate a concise justification in plain language.”. Do **not** trust the LLM’s arithmetic.

This division follows HackerRank advice that a lone LLM call is not enough [26†L131-L140] . A robust system calls the LLM as a *component*, not as the entire solution [26†L131-L140] [34†L165-L169] .

6. System Architecture Design

- **Time:** ~3–4 hours (high priority)
- **Deliverables:** Architecture diagram (mermaid), comparison table of approaches.
- **Risks:** Overcomplication; unclear data flow; missing validation/fallbacks.

6a. Proposed Architecture: We recommend a *hybrid pipeline* with clear modules (deterministic engine + AI interpreter). For example:

```
flowchart LR
  A[requests.csv] --> B[Request Processor]
  B --> C[Load User Profile & Events]
  C --> D[Financial Forecast Engine]
  B --> E[Context Collector]
  E --> F[Message Interpreter (LLM/OCR)]
  E --> G[Image OCR Handler]
  F --> D
  G --> D
  D --> H[Decision Engine]
  H --> I[Output CSV Writer]
```

This shows that each request is first loaded (`Request Processor`), then its financial data is gathered (`Load User Profile & Events`). Meanwhile, any messages/images are interpreted (via `Message Interpreter` and `Image OCR`). All context flows into a deterministic `Financial Forecast Engine` that computes safe amounts and schedules. Finally, the `Decision Engine` decides affordability, payment method, plan, and composes the row for `output.csv` .

6b. Alternative Architectures:

Approach	Pros	Cons / Risks
1. Single LLM Pipeline	Fast to prototype (one prompt handles everything).	<i>Very unsafe</i> : LLM can hallucinate financials; no audits. High risk of wrong output [26†L131-L140] .
2. Hybrid (Proposed)	Clear separation: code handles finances, LLM only for text. Deterministic and debuggable. Fits what judges expect [26†L131-L140] [34†L165-L169] .	More upfront coding work; need to orchestrate modules.
3. Modular Multi-Agent	Can break into sub-agents (e.g. one for forecasting, one for language). Scalable and testable by component.	Complex to manage communication; potential latency.
4. Rule-Based Only (No LLM)	Fully auditable; no hallucinations.	Likely cannot handle unstructured messages/images, explanation will be templated and flat.

The **hybrid approach** (row 2) is recommended. It aligns with past winners (“designing the loop” is crucial [34†L219-L223]) and avoids the high hallucination risk of an all-LLM solution [26†L131-L140] .

7. Implementation Plan (Code & Tools)

- **Time**: ~6–8 hours (high priority)
- **Deliverables**: Code repository structure, key scripts, libraries chosen.
- **Risks**: Poor code organization; missing dependencies; insufficient logging for transcripts.

7a. Development Environment: Use Python 3.x (for its strong data libraries). Key libraries: `pandas` (CSV I/O), `numpy` (calculations), `python-dateutil` (date math), `pytesseract` or `easyocr` (OCR), `openai` / `transformers` (LLM API calls), `spaCy` or `regex` (optional NER). Docker or venv for reproducibility.

7b. Folder Structure: A clean modular layout, for example:

```
/code
main.py          # Entry point to run end-to-end
loader.py        # Load CSVs (profiles, events, requests)
finance_engine.py # Deterministic logic (safe_amount, forecasting)
ai_module.py     # Functions to call LLM and process messages/images
```

```

    utils.py          # Utility functions (currency conversion, date parsing)
    prompts.py        # LLM prompt templates
  /tests
    test_finance.py   # Unit tests for deterministic logic
    test_ai.py        # Tests for message parsing (mocked)
  /dataset            # (Provided data in read-only)
  /output             # Where output.csv will be written
  README.md          # Setup instructions, how to run

```

Ensure the README explains setup (e.g., install packages, set API keys via environment variables) 【26†L119-L123】 【10†L328-L333】 .

7c. Chat Transcript Logging: Include code to capture all LLM interactions to a log file. HackerRank requires uploading the chat transcript 【10†L304-L314】 . For example: wrap LLM calls so that every prompt/response is appended to `log.txt` . (August Orchestrate had participants do exactly this 【10†L304-L314】 .)

7d. Testing Strategy:

- **Unit tests:** Write tests for core functions (safe amount calculation, forecasting, decision logic). Use known scenarios (e.g. from sample output) to assert correctness.
- **Sample check:** Run code on `sample_requests.csv` and manually verify the `output.csv` format and values match expectations. Use asserts to check column order and count match sample.

7e. Key Scripts:

- `main.py` should parse arguments, load data, loop over requests, call modules, and write `output.csv` .
- Use `if __name__ == "__main__":` block so it's runnable as a script.

Tools & Libraries Reference:

Python's pandas for CSV is standard. For LLM, any recent model (GPT-4, Claude 3, etc.) is acceptable. Use the model that the team is most comfortable with (prompt engineering is more important than model choice 【26†L125-L134】). OCR can use Tesseract (widely used open-source, e.g. 【38†L27-L34】) or an API.

8. Evaluation Strategy

- **Time:** ~2–3 hours (high priority)
- **Deliverables:** Checklist for verifying output, score estimation method.
- **Risks:** Mismatch in format; overlooked edge-case leading to zero points.

Action: Before submission, validate thoroughly:

- Check `output.csv` **format:** One row per request, exact column names/order. (HackerRank will automatically check this 【10†L328-L333】 .)
- **Compare with sample:** If a `sample_output.csv` is given, diff your results. If not, create a few hand-crafted examples.

- **Edge cases:** Test extreme cases (no balance, large expenses, cross-currency, missing data).
- **Safety check:** Ensure explanation strings are non-empty and reasons match decisions. Empty or contradictory reasons can hurt score [22†L159-L167] .

Evaluation Criteria (from Hackerrank): Remember that your solution will be graded on: accuracy of financial decisions, quality/usefulness of explanations, and overall safety. The blog notes that even with the right action, an empty or inconsistent explanation caps the score (~70) [22†L159-L167] . Also ensure “evidence” or justification is coherent if required. The behind-the-scenes rubric (for a support agent) emphasizes correct routing *and* consistent reasoning [22†L169-L173] , which applies analogously here.

9. AI Chat & Interview Preparation

- **Time:** ~2–3 hours (medium priority, spread out)
- **Deliverables:** Draft answers for likely questions; prepared chat snippets to demonstrate.
- **Risks:** Unfamiliarity with choices; inability to explain code decisions; AI hallucination in transcript.

Chat Transcript Tips: During development, **lead the conversation** with the AI agent. The chat log is graded on demonstrating **ownership and technical precision** [22†L185-L194] . For example, when using the LLM, prompt with specific instructions:

- Define strict JSON schemas for LLM outputs, enforce “return only JSON with these fields.” (Shows structure constraint.)
- “If the content is not relevant, respond with ‘no_finance_info’.” (Shows guardrails.)

AI Judge Interview: Be ready to discuss and defend your design:

- Explain the high-level **architecture and data flow**.
- Justify why code (not LLM) handles calculations (as per Mrinal Desai’s advice [34†L165-L169]). Mention that numeric safety is provably correct by code.
- Discuss failure modes: e.g. “If the LLM misinterprets a message, we fall back on direct regex or ignore ambiguous info. We also validate all numeric outputs.”
- Show you understand the **evaluation rubric**: Emphasize how you tested outputs and explanation quality (Direction & iteration) [22†L185-L194] [22†L193-L199] .
- Common questions to practice: Why not just ask GPT to compute? (Answer: it’s unreliable and non-deterministic [34†L198-L203] .) How did you choose payment plans? (Show the criteria you used.) How would you handle multi-currency? (Explain using `exchange_rates.csv`.)

Sample **chat prompt** for explanation generation:

User wants to buy X for \$70,000. Current safe funds are \$35,000 and future salary will arrive on Sep30.
 Prompt: "Based on these facts, write a clear recommendation whether to buy now or wait."

This ensures the LLM response uses your computed facts.

10. Submission Checklist

- **Time:** ~1 hour (final step)
- **Deliverables:** Final code zip, `output.csv`, `log.txt`.
- **Risks:** Missing files, formatting errors.

Before submitting, confirm:

- **Code Zip:** Contains all source code, a README with instructions, and any config (prompt templates, environment variables listing). (Do **not** include any API keys.)
- `output.csv`: Has one row per request and *only* the required columns (in exact order). No extra columns or missing commas 【10†L328-L333】.
- **Chat Transcript** (`log.txt`): Captures all LLM interactions (prompts and responses) in chronological order. You can automate logging in code 【10†L304-L314】.
- **Transcript sanity:** Remove any sensitive info. Ensure the transcript shows you actively guiding the AI (not blind copying of chatGPT output).

Finally, double-check you haven't hard-coded any answers (the organizers disallow using the hidden ground truth) 【10†L328-L333】.

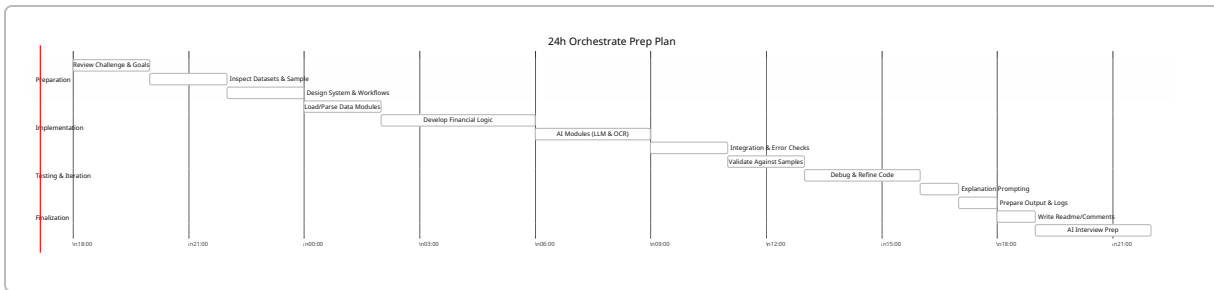
11. Resume & PR Tips

Highlight This Project: If you place well, note it on your resume as “*HackerRank Orchestrate (Sep 2026) — [Winner/Top N]*”, with a short bullet like “*Designed an AI agent for personal finance planning: integrated structured data (budgets, transactions) with LLM reasoning for recommendations*”. Even without a top rank, mention the challenge participation under “Projects” or “Achievements” with **emphasis on skills**: AI agent design, system architecture, prompt engineering. HackerRank provides a certificate of participation 【1†L53-L57】; however, the real resume value comes from describing your technical accomplishment (the code and reasoning) and, ideally, a good placement.

Celebrate Results: If you rank highly, you can post on LinkedIn/GitHub with your solution code (sanitize any private info). Show the **architecture diagram** and some sample decisions from your agent. Recruiters and tech peers are impressed by “agentic AI” projects. Remember the Orchestrate grading criteria: it’s a measurable engineering achievement, not just “I used AI” 【34†L219-L223】. Frame it as solving a concrete problem with hybrid AI+software.

Timeline (24–72 Hour Sprint)

Below is an example Gantt chart for a **24-hour to 3-day sprint**. Adjust durations as needed:



Core Formulas & Pseudocode Examples

For clarity, here are **example formulas/pseudocode** used above:

- **Safe Amount:**

```
safe_now = max(0, balance - min_balance -
sum(upcoming_essentials_due_within_month))
```

- **Affordability Status Logic (simplified):**

```
if safe_now >= cost:
    status = "affordable_now"; method="full_payment"
elif exists(partial_option) and safe_now >= partial_option.upfront:
    status = "affordable_with_plan"; method="partial_payment"
elif exists(installment_plan) and installments_feasible:
    status = "affordable_with_plan"; method="installments"
elif future_date_found:
    status = "affordable_later"; method="wait"
else:
    status = "not_affordable"; method="not_recommended"
```

- **Forecast Loop:**

```
balance = current_balance
for date, change in sorted(future_events):
    balance += change
    if balance - min_balance >= cost:
        earliest_date = date
        break
```

- **LLM Prompt (explanation):**

```
"User can pay up to ₹35,000 safely now, but item costs ₹70,000. Recommend action:"
```

Each of the above is executed in code; only the final human-readable **reason** is given to the LLM as context for wording.

Conclusion

This rigorous plan covers all aspects needed for a first-time Orchestrate participant. Following it will help you build a reliable agent: **validate every step with code and tests**, rely on determinism for the critical financial logic, and use the LLM judiciously for interpretation and explanation. Keep the HackerRank evaluation criteria in mind: correct outputs and coherent reasoning are paramount [22†L159-L167]. With disciplined execution (and a bit of endurance over the 24–72 hours), you'll have a submission ready to impress the AI Judge and potentially earn top rank — a notable achievement for your resume. Good luck!

Sources: HackerRank Orchestrate documentation and blog posts [26†L72-L80] [26†L131-L140] [10†L259-L267] [10†L328-L333] [22†L159-L167] [22†L185-L194] [34†L165-L169] [34†L219-L223] provide guidance on system design, evaluation metrics, and agent best practices.
